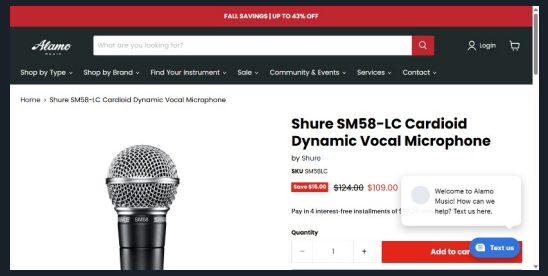
How the agent works: observe → decide → act → verify

Real step 2 from our run (openai/gpt-4.1, browser-use 0.13.11), headed Chrome, fresh profile

1 OBSERVE



Screenshot + page elements the agent saw

2 DECIDE

Goal:
Scroll down the page to search for shipping cost information or any note about free shipping.

Reasoning:
I have navigated to the product page for the Shure SM58-LC Cardioid Dynamic Vocal Microphone on alamomusic.com. The product title and SKU match the requested model (SM58-LC, new, no cable). The price is clearly shown as $109.00. I need to check the page for shipping information—whether shipping is ...

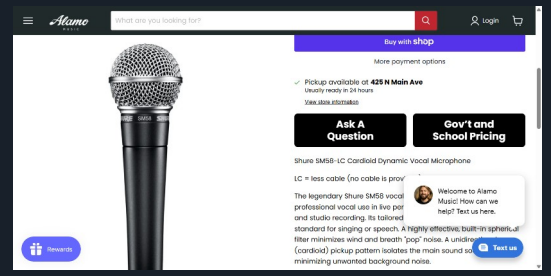
3 ACT

Action(s): scroll

[{"scroll": {"down": true, "pages": 1.0}}]

(timing for each step is in the report)

4 VERIFY



Same agent, next step: did it work?
Agent says: Scroll action succeeded, but no explicit shipping cost or free shipping note is visible yet.

Same task, two approaches: measured

Mean of repeated runs; success = every item returned and its price re-verified on the page

Task	Agent time	Agent cost	Agent requests	Agent success	Agent consistency	Det. time	Det. cost	Det. requests	Det. success	Det. consistency
Fallback: 3 books	73 s	$0.138	9 model calls + 9 page loads	0/3	3/3 identical	4.8 s	$0	6 HTTP requests	3/3	3/3 identical
Live: 3 shops	54 s	$0.068	5 model calls + 4 page loads	0/3	3/3 identical	1.7 s	$0	2 HTTP requests	0/3	3/3 identical

- Agent: prices right 9/9 on the books, but it reported a product URL it never visited in 3/3 runs, so strict success is 0/3. Our independent re-fetch caught it.
- Live: both approaches lost one shop. Sweetwater showed a human-verification page: the agent was stopped without touching it, plain HTTP got 403. Two shops verified OK.
- Deterministic: fast, free, repeatable, but one extractor per site, breaks if layout changes, and cannot see shipping that is not in the static page.
- Honest limits: 3 runs, one model (gpt-4.1), one day; the check re-reads pages with a deterministic parser.

When to use a browser agent, and when to use an API

Use a browser agent when:

- there is no API or data feed
- many different sites, layouts you do not control
- the answer is only visible after rendering or interaction
- one-off or low-volume research tasks

Guard it: allow-list, step and time limits, read-only, verify the output

Use an API or structured data when:

- a documented API or feed exists
- the same sites are read again and again
- you need speed, low cost, exact repeatability
- results feed other systems or decisions

Expect: one extractor per site, maintenance when pages change

Best of both: let the agent explore and extract once, then turn what it learned into a deterministic extractor for the sites you keep using.
Never trust an agent's own 'done': check the numbers independently (our check caught a made-up product URL in every fallback run).